

Vulnerability Assessment Report

Project: SQL Injection Vulnerability Assessment

Target: PortSwigger Web Security Academy

Prepared by: Kumavath Tanishq (Intern ID: CA0593)

1. Executive Summary

This assessment was performed on the PortSwigger Web Security Academy lab 'SQL injection vulnerability in WHERE clause allowing retrieval of hidden data'. The objective was to identify and validate a SQL injection vulnerability in the product category filter. Testing confirmed that the application was vulnerable to SQL injection and allowed retrieval of hidden products.

2. Objective

Identify, verify, and document a SQL injection vulnerability affecting the category parameter of the application.

3. Scope

Target: PortSwigger Web Security Academy SQL Injection Lab

Parameter Tested: category

Method: Manual testing using Burp Suite Professional.

4. Tools Used

1. Burp Suite Professional
2. Mozilla Firefox
3. PortSwigger Web Security Academy

5. Methodology

1. Opened the SQL Injection lab.
2. Captured the GET request using Burp Suite Proxy.
3. Sent the request to Repeater.
4. Modified the category parameter by adding a single quote (').
5. Observed HTTP 500 Internal Server Error indicating SQL syntax interference.
6. Modified the request using the payload:
' OR 1=1--+

7. Received HTTP 200 OK.
8. Verified that hidden products were displayed.
9. Confirmed successful completion of the lab.

6. Vulnerability Details

Vulnerability: SQL Injection

Severity: High

Original request: GET /filter?category=Pets

Detection payload: GET /filter?category=Pets'

Exploitation payload: GET /filter?category=Pets'+OR+1=1--+

The payload altered the SQL query logic by injecting an always-true condition and commenting out the remaining query.

7. Proof of Concept

Figure 1 – 01_Lab_Description.png

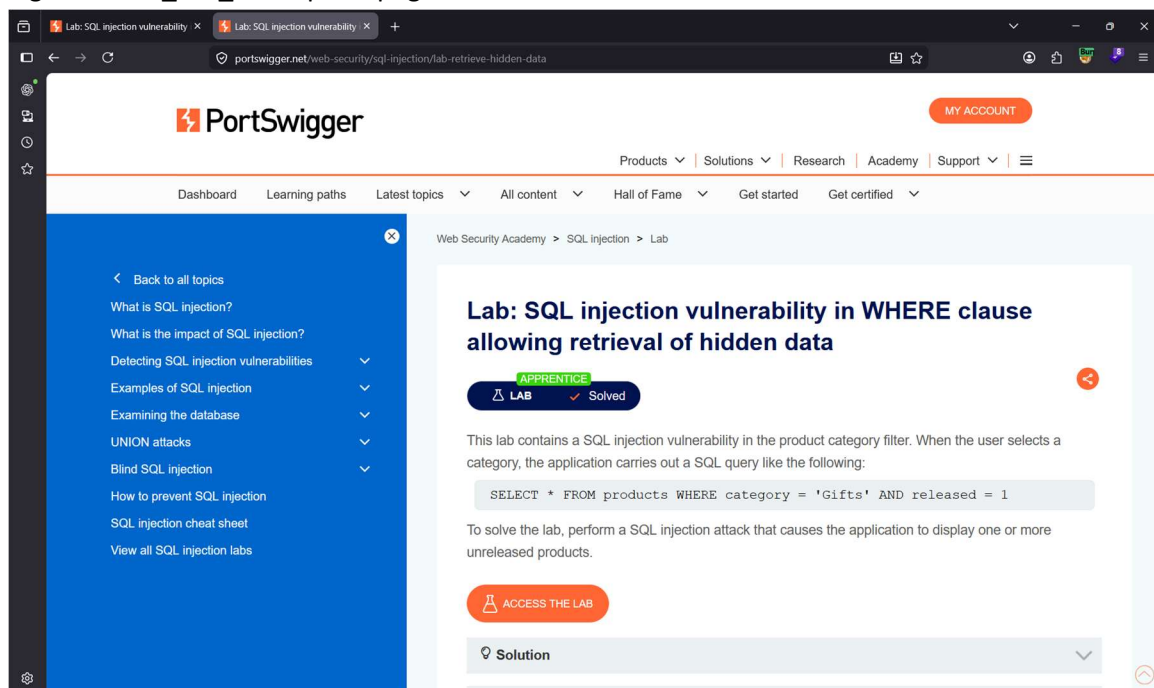


Figure 2 – 02_HTTP_History.png

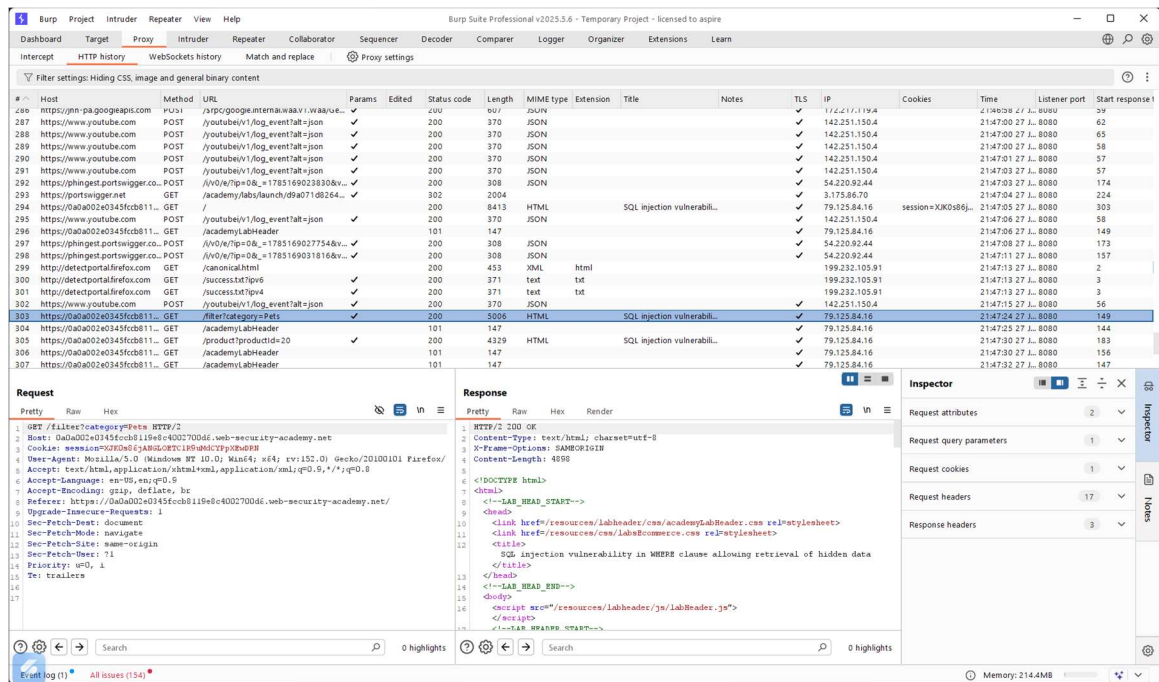


Figure 3 – 03_SQL_Error_500.png

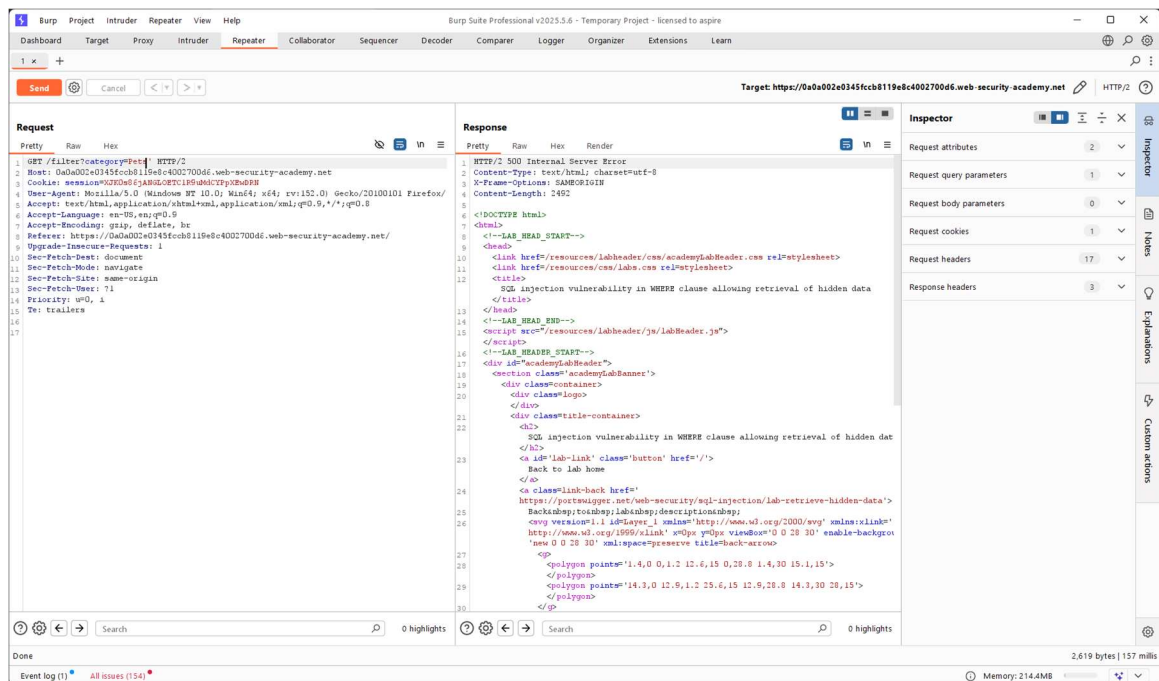


Figure 4 – 04_SQLi_Repeater_Success.png

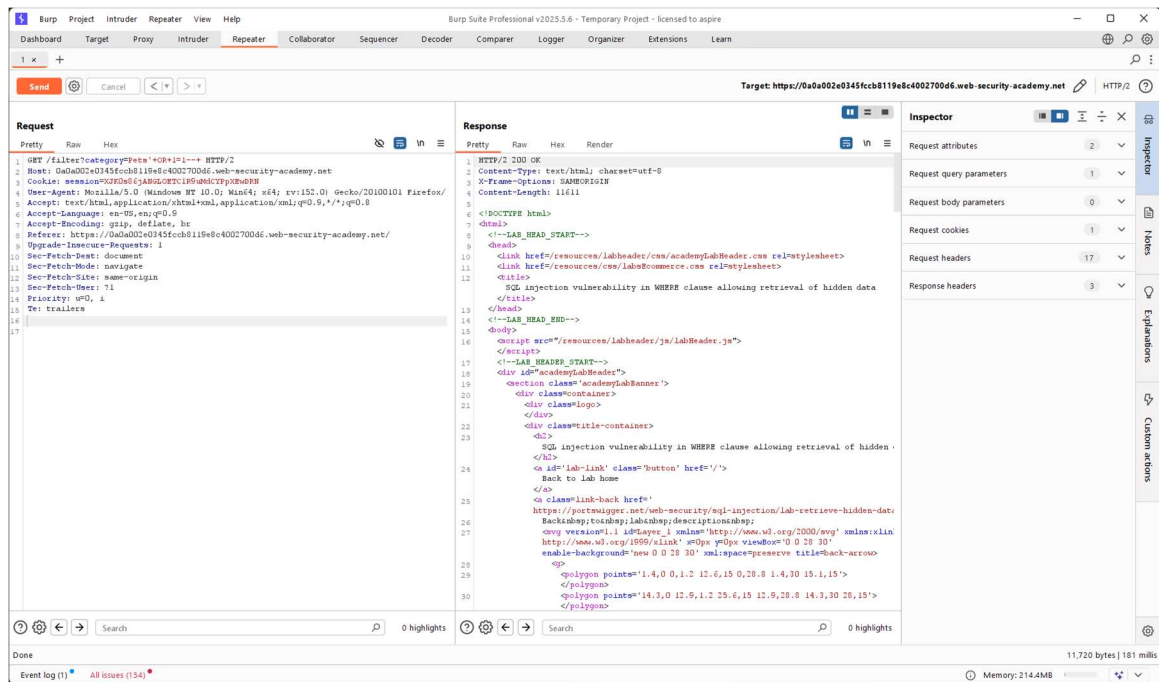
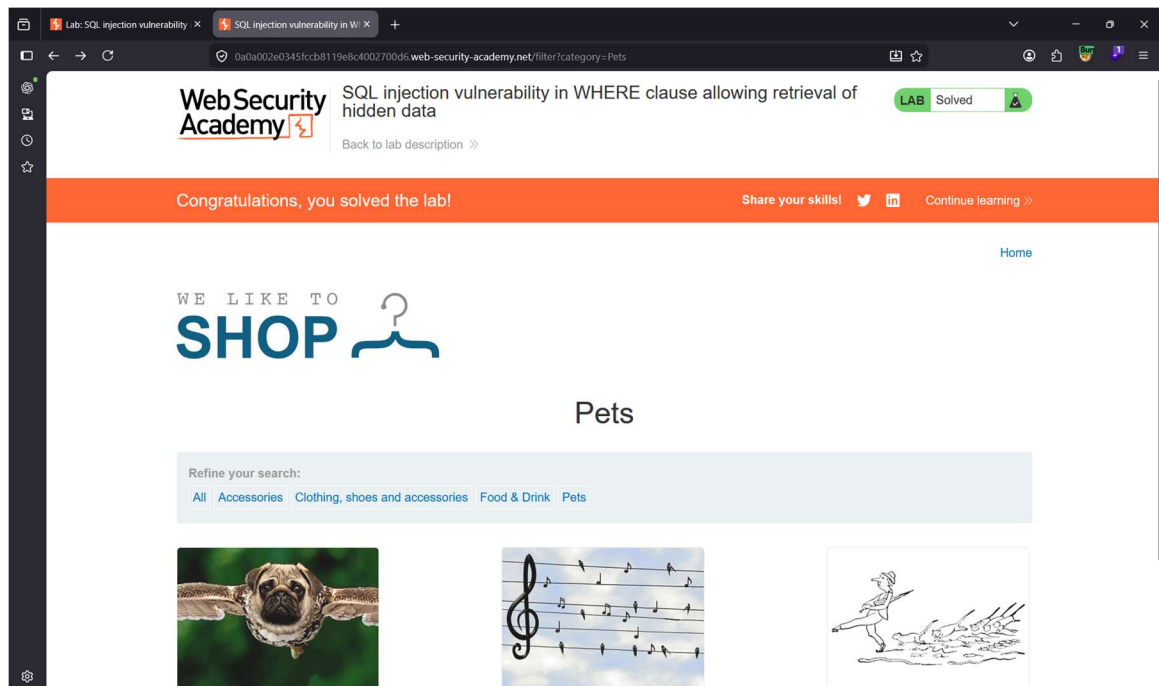


Figure 5 – 05_Lab_Solved.png



8. Impact

If exploited in a real application, this vulnerability could:

- Expose sensitive information.
- Retrieve hidden or unauthorized data.
- Bypass application logic.
- Potentially enable modification or deletion of database records depending on permissions.

9. Risk Assessment

Risk Level: High

Likelihood: High

Impact: High

SQL Injection is one of the most critical web vulnerabilities because it can compromise the confidentiality, integrity, and availability of application data.

10. Remediation

- Use parameterized queries (prepared statements).
- Validate and sanitize user input.
- Avoid dynamic SQL query construction.
- Apply least-privilege access for database accounts.
- Implement secure error handling.
- Conduct regular security testing.

11. Conclusion

The assessment confirmed that the category parameter was vulnerable to SQL injection. By exploiting the vulnerability with the payload ' OR 1=1--+, hidden products were successfully retrieved and the PortSwigger lab was marked as solved. Proper input validation and parameterized queries are recommended to mitigate this issue.

12. References

- PortSwigger Web Security Academy
- OWASP SQL Injection Prevention Cheat Sheet
- OWASP Top 10